Universidad Católica Boliviana “San Pablo”
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas

La Paz - Bolivia

Desarrollo de un Sistema de Información Organizacional Web basado en el enfoque de Procesamiento de Transacciones (TPS) para la gestión de procesos, usuarios, transacciones y reportes para una cafetería

Integrantes:

- Claros Suntura Juan Jose *(Desarrollador FullStack)*
- Lecoña Condori Elias Milan *(Desarrollador FullStack)*
- Maldonado Carvajal Alan Ariel *(Scrum Master)*
- Torrez Calle Alvaro Ariel *(Product Owner)*

Materia: Ingenieria de Software (SIS-213)
Docente: Miguel Angel Pacheco Arteaga
Fecha: 10/04/2026
Universidad: Universidad Católica Boliviana

Índice General

Capítulo 1. MARCO REFERENCIAL DEL SISTEMA TPS **1**

1.1 Introducción 1

1.2 Antecedentes 2

1.2.1 Antecedentes del objeto de estudio 2

1.2.2 Referencias técnicas de otros sistemas TPS 3

1.3 Descripción del objeto de estudio 4

1.4 Identificación del Problema 5

1.5 Formulación del Problema 6

1.6 Objetivos 6

1.6.1 Objetivo General 6

1.6.2 Objetivos Específicos 7

1.7 Justificación 7

1.7.1 Justificación Técnica 7

1.7.2 Justificación Organizacional 8

1.7.3 Justificación Económica 8

1.8 Límites y Alcances 9

1.8.1 Límites 9

1.8.2 Alcances 9

1.9 Metodología del Proyecto 9

1.9.1 Tipo de estudio 9

1.9.2 Metodología de desarrollo 10

1.9.3 Técnicas 10

1.10 Análisis preliminar del sistema TPS 11

1.11 Propuesta de solución 12

1.12 Cronograma 14

Capítulo 2. MARCO TEÓRICO DEL SISTEMA TPS **17**

2.1 Sistemas de Información Organizacional 17

2.1.1 Definición 17
2.1.2 Componentes y Tipos 17
2.1.3 Importancia en organizaciones del sector gastronómico 18
2.2 Sistema de Procesamiento de Transacciones (TPS) 19
2.2.1 Características principales 19
2.2.2 Funciones del TPS en el sistema POS de la cafetería 20
2.2.3 Evolución hacia sistemas web 20
2.3 Arquitectura de sistemas web 21
2.4 Seguridad en sistemas de información 21
2.5 Base de datos 22
2.6 Metodología de desarrollo 23
2.6.1 Roles 23
2.6.2 Artefactos 24
2.6.3 Eventos 26

Capítulo 3. MARCO PRÁCTICO — DESARROLLO DEL SISTEMA TPS 27
3.1 Análisis del sistema 27
3.2 Determinación de requerimientos 28
3.2.1 Requerimientos funcionales 28
3.2.2 Requerimientos no funcionales 28
3.3 Modelado del sistema 29
3.3.1 Historias de Usuario 29
3.3.2 Diagramas UML 29
3.4 Diseño del sistema 30
3.4.1 Arquitectura del sistema 30
3.5 Diseño de la Base de Datos 30
3.6 IMPLEMENTACIÓN DE LOS MÓDULOS DEL SISTEMA 31
3.6.1 Módulo de Catálogo, Menú y Mesas 31
3.6.2 Módulo de Usuarios y Roles 31
3.6.3 Módulo de Transacciones (El Punto de Venta) 31

3.6.4 Módulo de Reportes 32
3.7 Capa Backend Funcional 32
3.8 Validación y pruebas del sistema 32
3.9 Desarrollo del prototipo funcional 33
3.10 Documentación de Ingeniería Completa 33

Capítulo 4. CONCLUSIONES Y RECOMENDACIONES 34

4.1 Conclusiones 34
4.2 Recomendaciones 34

Índice de Figuras, Tablas y Diagramas

Figuras

1 Ejemplo de prueba en el análisis del sistema 27

Tablas

1 Tecnologías seleccionadas (Pila MERN) 13
2 Cronograma de Sprints 16
3 Product Backlog Inicial 25
4 Diccionario para colección de Órdenes (Base de Datos) 30

Diagramas

1 Diagrama de Arquitectura MERN de tres capas del Sistema POS . . . 14
2 Ejemplo general de Diagrama Estructural UML 29

Capítulo 1. MARCO REFERENCIAL DEL SISTEMA TPS

1.1 Introducción

En el contexto actual de la transformación digital, las organizaciones de todo tamaño enfrentan la necesidad imperiosa de modernizar sus procesos operativos. Los Sistemas de Información Organizacional (SIO) han emergido como la respuesta tecnológica a esta necesidad, permitiendo a las empresas capturar, procesar y distribuir información de manera eficiente, precisa y centralizada. Dentro de esta categoría, los Sistemas de Procesamiento de Transacciones (TPS, por sus siglas en inglés — *Transaction Processing Systems*) representan el eslabón más fundamental: son la capa operativa que registra y procesa cada evento del negocio en tiempo real, constituyendo la base sobre la que se construyen todos los demás niveles de información organizacional.

La evolución histórica de los TPS es un reflejo directo del avance tecnológico. En sus orígenes, durante las décadas de 1960 y 1970, estos sistemas operaban en mainframes centralizados con procesamiento por lotes (*batch processing*), donde las transacciones se acumulaban y procesaban en diferido. Con la llegada de las redes de computadoras y, posteriormente, de Internet, los TPS evolucionaron hacia arquitecturas distribuidas de procesamiento en línea (*OLTP — Online Transaction Processing*), capaces de manejar miles de transacciones concurrentes con tiempos de respuesta de milisegundos. Hoy en día, la adopción de arquitecturas web modernas basadas en *stacks* como MERN (MongoDB, Express.js, React.js, Node.js) ha democratizado la implementación de TPS robustos, haciéndolos accesibles incluso para pequeñas y medianas empresas que anteriormente no podían costear soluciones de este tipo.

En este marco, el presente proyecto propone el desarrollo de un Sistema de Información Organizacional Web basado en el enfoque TPS, orientado específicamente a la gestión integral de una cafetería en la ciudad de La Paz, Bolivia. El sistema, concebido como un Punto de Venta (*Point of Sale* — POS) web, tiene por objetivo digitalizar y centralizar los procesos críticos del negocio: la toma y seguimiento de órdenes por mesa,

la administración del catálogo de productos, el control de acceso diferenciado por roles de usuario, el procesamiento de cobros y la generación automatizada de reportes financieros. La solución busca reemplazar los procesos manuales actualmente vigentes — basados en libretas de papel, cálculos mentales y ausencia total de trazabilidad — por una plataforma tecnológica accesible, segura y escalable que eleve la eficiencia operativa del establecimiento.

1.2 Antecedentes

1.2.1 Antecedentes del objeto de estudio

El objeto de estudio del presente proyecto es una cafetería de tamaño mediano en la ciudad de La Paz, Bolivia, que actualmente opera sus procesos de atención y venta de manera completamente manual. Este escenario, lejos de ser un caso aislado, refleja la realidad predominante en el sector gastronómico local, donde la adopción de tecnologías de gestión sigue siendo incipiente.

En su estado actual, el flujo operativo de la cafetería depende íntegramente de la intervención humana no sistematizada: los pedidos de los clientes son anotados a mano por el personal en libretas o talonarios de papel, la comunicación entre el área de atención y la barra de preparación se realiza de forma verbal o mediante la entrega física de las comandas, y el proceso de cobro se ejecuta mediante cálculos mentales o con el apoyo de calculadoras de escritorio. Al cierre de cada jornada, el arqueo de caja se realiza de forma manual, contrastando el efectivo físico con las anotaciones del día, un proceso propenso a errores y discrepancias.

Esta situación genera un conjunto de problemas operativos concretos y recurrentes que afectan tanto la eficiencia del servicio como la salud financiera del negocio:

- **Pérdida e ilegibilidad de comandas:** El registro manual en papel está expuesto a extravíos, manchas o escritura ilegible, lo que deriva en errores en la preparación de pedidos y conflictos con los clientes.
- **Ausencia de control y auditoría de usuarios:** Al no existir un sistema de reg-

istro, resulta imposible determinar qué miembro del personal procesó, modificó o anuló una orden, eliminando cualquier posibilidad de rendición de cuentas.

- **Descuadres de caja recurrentes:** El cobro basado en cálculo mental o en calculadoras simples, sin un sistema que valide automáticamente los totales, genera discrepancias diarias entre los ingresos registrados y el efectivo real.
- **Nula trazabilidad histórica:** La ausencia de una base de datos implica que no existe ningún registro histórico de ventas. La gerencia no puede conocer cuáles son los productos más vendidos, los picos de demanda por hora o los ingresos acumulados por período, privándose de información crítica para la toma de decisiones estratégicas.

1.2.2 Referencias técnicas de otros sistemas TPS

En el contexto local de La Paz, Bolivia, existe una notable carencia de sistemas TPS especializados y accesibles para el rubro de las cafeterías, o en su defecto, hay un desconocimiento generalizado sobre soluciones de *software* a medida. Por lo tanto, como Ingeniero de Requerimientos y Datos, el análisis de referencia se basa en el modelo transaccional “tradicional” o empírico que emplean actualmente la mayoría de las cafeterías en la ciudad sin un sistema digital centralizado:

1. **Sistema analógico de comandas y caja básica**
    - **Tipo de sistema:** TPS manual y físico.
    - **Funcionalidades principales:** El mesero anota el pedido y el número de mesa en una libreta de papel (comanda), el cual se entrega físicamente a la barra de preparación. El cobro se realiza calculando mentalmente o con una calculadora de escritorio, guardando el dinero en una caja registradora simple o gaveta de efectivo.
    - **Diferencia con el sistema propuesto:** Este método empírico carece de cualquier tipo de integridad de datos. Nuestro sistema de *software* digitalizará la entrada de la orden y la asignación de mesas en tiempo real, eliminando errores de cálculo, previniendo la pérdida de comandas de papel

y asegurando que cada producto despachado quede registrado inmutablemente en la base de datos para un arqueo de caja exacto.

2. **Registro diferido en hojas de cálculo (Excel)**
   - **Tipo de sistema:** TPS semiautomatizado y no concurrente.
   - **Funcionalidades principales:** Al final de la jornada, el administrador recolecta los apuntes manuales e intenta cuadrar los ingresos del día transcribiéndolos a una hoja de cálculo.
   - **Diferencia con el sistema propuesto:** Una hoja de cálculo no es una base de datos transaccional ni opera en tiempo real. El sistema web propuesto validará cada transacción en el momento exacto del cobro de la mesa, guardando relaciones seguras y automáticas entre el cajero en turno, los productos vendidos y la fecha exacta de emisión.

1.3 Descripción del objeto de estudio

La unidad de análisis del presente proyecto es una cafetería de servicio rápido ubicada en la ciudad de La Paz, Bolivia, con capacidad para atender simultáneamente a múltiples mesas. El establecimiento ofrece un menú compuesto principalmente por bebidas calientes y frías (cafés, infusiones, batidos) y una selección de alimentos de preparación rápida (postres, sándwiches, snacks), atendiendo a una clientela diversa en horario continuo.

Estructura organizacional y actores del sistema:

El personal operativo del establecimiento se organiza en dos roles funcionales claramente diferenciados, que se corresponden directamente con los actores del sistema a desarrollar:

- **Administrador:** Responsable de la gobernanza general del negocio. Gestiona el catálogo de productos (altas, bajas y modificaciones de precios), administra las cuentas del personal operativo y tiene acceso a los reportes de ventas e indicadores financieros.

- **Cajero / Operador POS:** Personal de atención directa al cliente. Su función central es construir y procesar órdenes en la interfaz del punto de venta, asignarlas a la mesa correspondiente y ejecutar el cobro al momento de cerrar la cuenta.

Flujo actual del negocio (situación sin sistema):

El ciclo de servicio actual sigue el siguiente flujo manual: el cliente se ubica en una mesa disponible → el cajero toma el pedido verbalmente y lo anota en la comanda de papel → la comanda se entrega en barra para preparación → los productos son entregados al cliente → al solicitar la cuenta, el cajero suma manualmente los ítems, comunica el total y recibe el pago en efectivo → el dinero se deposita en la caja registradora mecánica.

Flujo propuesto del negocio (con el sistema TPS):

Con la implementación del sistema, el flujo se transforma radicalmente: el cajero selecciona los productos del menú digital interactivo y los asigna a la mesa del cliente en la interfaz POS → el sistema calcula automáticamente el subtotal en tiempo real → al confirmar el cobro, el *backend* procesa matemáticamente la transacción, aplica impuestos y sella el registro de forma inmutable en la base de datos → el sistema genera automáticamente el comprobante de pago (ticket/factura) → la mesa queda marcada como disponible para el próximo cliente. Cada uno de estos eventos queda registrado con fecha, hora, cajero responsable y detalle de productos, garantizando trazabilidad completa y eliminando cualquier posibilidad de descuadre.

1.4 Identificación del Problema

La cafetería objeto de estudio opera actualmente sin ningún sistema de información digital que centralice y automatice sus procesos transaccionales. Esta carencia se manifiesta en cinco dimensiones de problema interrelacionadas:

En primer lugar, la **ineficiencia operativa en la toma y seguimiento de pedidos**: la gestión manual de comandas en papel provoca errores en la preparación, pedidos duplicados u omitidos, y tiempos de espera prolongados para el cliente, especialmente durante los picos de demanda. En segundo lugar, la **imposibilidad de control y audi-**

toría de usuarios: sin un sistema de registro de sesiones, es imposible determinar qué operario procesó, anuló o modificó una orden, creando un entorno sin rendición de cuentas. En tercer lugar, la **inexactitud en el cobro y los arqueos de caja**: la dependencia del cálculo mental y de las calculadoras genera descuadres frecuentes entre el registro de ventas y el efectivo físico al cierre del turno. En cuarto lugar, la **ausencia total de datos históricos**: sin una base de datos, la gerencia carece de información sobre ventas por período, productos más demandados o rendimiento del personal, limitando severamente su capacidad de tomar decisiones informadas. Finalmente, la **falta de escalabilidad del modelo operativo actual**: el sistema manual no puede adaptarse a un eventual crecimiento del negocio (más mesas, más personal, múltiples sucursales) sin incrementar proporcionalmente los errores y la carga operativa.

1.5 Formulación del Problema

¿De qué manera el desarrollo e implementación de un Sistema de Información Organizacional Web basado en el enfoque de Procesamiento de Transacciones (TPS), construido sobre la arquitectura MERN, puede optimizar la gestión de pedidos, la administración de mesas, el control de usuarios por roles, el procesamiento de cobros y la generación de reportes financieros en una cafetería de la ciudad de La Paz, reduciendo los errores operativos y garantizando la trazabilidad e integridad de cada transacción?

1.6 Objetivos

1.6.1 Objetivo General

Desarrollar un Sistema Web de Punto de Venta (POS) para cafeterías, basado en la arquitectura MERN (MongoDB, Express.js, React.js, Node.js), que permita optimizar la gestión integral de pedidos, la administración visual de mesas y la facturación automatizada, reduciendo los errores operativos y mejorando la experiencia del cliente en establecimientos gastronómicos medianos de la ciudad de La Paz.

1.6.2 Objetivos Específicos

- Implementar un módulo de gestión de pedidos en tiempo real que permita a los meseros registrar, modificar y hacer seguimiento del estado de las órdenes (en preparación, servido, pagado) mediante una interfaz táctil dinámica construida con React.js.
- Diseñar e integrar un sistema de control de acceso basado en roles (RBAC) con autenticación segura mediante JSON Web Tokens (JWT), diferenciando los permisos y vistas del Administrador, el Cajero y el Mesero.
- Desarrollar un módulo de administración visual de mesas y reservas que presente un panel de estados en tiempo real, permitiendo a los operadores conocer la disponibilidad y el estado de cada mesa del establecimiento.
- Construir las APIs RESTful del *backend* con Node.js y Express.js, conectadas a una base de datos MongoDB, para soportar todas las operaciones CRUD de los módulos de productos, órdenes, mesas y usuarios.
- Implementar un módulo de facturación y generación de recibos que automatice el cálculo del cobro total y produzca comprobantes detallados en formato PDF, integrando métodos de pago simulados mediante una pasarela estándar.
- Desplegar el sistema en infraestructura *cloud* (AWS o DigitalOcean) utilizando contenedores Docker para garantizar la portabilidad, disponibilidad y escalabilidad del entorno productivo.

1.7 Justificación

1.7.1 Justificación Técnica

Desde el rol de ingeniería de requerimientos y datos, la implementación de este sistema en una cafetería que opera de forma 100% manual (con libretas de papel y sin ningún tipo de *software*) se justifica por la necesidad urgente de establecer un núcleo de persistencia de datos robusto y seguro. Se diseñará una base de datos estructurada

(adoptando el paradigma de la pila MERN, como MongoDB) que reemplace la vulnerabilidad de los cuadernos físicos. Esto garantiza la integridad referencial de la información, conectando de forma exacta el catálogo de productos con el carrito de ventas, el número de mesa y el usuario que procesa el cobro. El *backend* (Node.js/Express) manejará de forma atómica cada transacción, mientras que la seguridad se blindará mediante encriptación de contraseñas y autenticación por *tokens* (JWT), asegurando la protección de los datos financieros ante cualquier interrupción o alta concurrencia.

1.7.2 Justificación Organizacional

Actualmente, el uso exclusivo de comandas escritas a mano y la comunicación verbal generan un caos organizativo constante: los meseros equivocan las mesas, los pedidos se pierden o resultan ilegibles para el área de preparación, y no hay control sobre quién realiza qué cobro. La implementación de este TPS estandarizará el flujo operativo. A nivel de requerimientos, el sistema forzará jerarquías claras de acceso (Administrador y Cajero) y digitalizará la selección de productos y la asignación específica de las mesas. Esto elimina de raíz la asimetría de información, asegurando que el estado de cada orden, desde que se toma el pedido en la mesa hasta que se imprime el ticket final, sea transparente y rastreable en tiempo real para todo el personal autorizado.

1.7.3 Justificación Económica

La dependencia del cálculo mental y de anotaciones manuales para cobrar a los clientes y realizar el arqueo de caja es altamente susceptible al error humano. Estas fallas diarias se traducen en cobros incompletos, descuadres de caja y una pérdida monetaria silenciosa y constante para la cafetería. Al transicionar a un motor transaccional de Punto de Venta (POS) centralizado —que calcula automáticamente los subtotales, impuestos y totales finales antes de generar la factura— se erradican las fugas de capital por malas sumas. Además, la centralización de los datos permitirá a la gerencia consultar reportes históricos exactos (como los productos más vendidos), facilitando una toma de decisiones inteligente y la optimización del presupuesto para la compra de

insumos.

1.8 Límites y Alcances

1.8.1 Límites

El sistema:

- Será exclusivamente web (no incluirá aplicaciones móviles nativas).
- Usará una base de datos NoSQL orientada a documentos enfocada en transacciones rápidas.
- Tendrá un sistema cerrado de autenticación de usuarios locales (no integrará inicios de sesión sociales).
- No incluirá integración directa a pasarelas de pago bancarias físicas en su primera versión (los pagos se registrarán de forma lógica).

1.8.2 Alcances

El sistema permitirá:

- Gestionar el catálogo interactivo de la cafetería (categorías, productos, precios).
- Gestionar usuarios, asignar roles y administrar permisos granulares mediante *middlewares*.
- Registrar transacciones en el POS vinculando carritos de compra a mesas específicas.
- Generar reportes tabulares y calcular cortes de caja.

1.9 Metodología del Proyecto

1.9.1 Tipo de estudio

Por su naturaleza, la investigación será de tipo **Aplicado** (orientado a la solución de un problema concreto identificado en cafeterías de La Paz), **Tecnológico** (mediante el

diseño e implementación de una plataforma *software*) y **Descriptivo** (para modelar el escenario y el comportamiento actual de los procesos manuales del negocio).

El enfoque es **mixto**: cuantitativo en la medición del esfuerzo mediante la técnica de Puntos de Función COSMIC y la estimación presupuestaria; y cualitativo en el levantamiento de requerimientos con los actores del negocio a través de entrevistas y observación directa.

1.9.2 Metodología de desarrollo

Se aplicará la metodología ágil ***Scrum***, que se adapta idealmente al desarrollo de *software* iterativo e incremental:

- ***Sprints*** (Iteraciones): Ciclos de trabajo de 2 semanas de duración.
- ***Product Backlog*** (Pila de producto): Lista priorizada de todos los requerimientos y módulos del sistema.
- ***Sprint Backlog*** (Pila del ciclo): Tareas seleccionadas para ser desarrolladas en el *Sprint* actual.
- **Entregables:** Incrementos funcionales al final de cada iteración, demostrando valor medible.

1.9.3 Técnicas

- **Entrevistas:** Al personal clave de la cafetería para levantar requerimientos operativos y de negocio.
- **Observación directa:** De los procesos manuales actuales en el sitio (toma de comandas, cobro, asignación de mesas).
- **Modelado de Datos:** Estructuración de colecciones JSON y flujos de API REST.
- **Modelado UML:** Para representar gráficamente el diseño de la solución (casos de uso, diagramas de clase, secuencia y despliegue).
- **Puntos de Función COSMIC:** Para la medición objetiva del tamaño funcional del *software* y su estimación de costos.

1.10 Análisis preliminar del sistema TPS

Como Ingeniero de Requerimientos y Datos, el análisis preliminar dictamina que la transición de una cafetería puramente manual a un entorno digital requiere modelar transacciones rápidas y precisas, integrando la gestión física del local (mesas). Se han especificado los siguientes requerimientos de persistencia y flujo:

- **Detalle de Usuarios (Actores y Control de Acceso):**
  - **Administrador:** Actor con privilegios absolutos (*Full CRUD*). Su rol a nivel de datos implica alimentar las entidades maestras del sistema: gestión del catálogo de ítems de la cafetería (nombres, precios, imágenes, categorías como 'Cafés', 'Postres'), control de inventario base y la creación o inhabilitación de cuentas para el personal operativo.
  - **Cajero / Operador POS:** Actor confinado a la interfaz de Punto de Venta. Su interacción con la base de datos es de lectura sobre el menú y de escritura intensiva sobre las colecciones/tablas de ventas, sin permisos para alterar precios históricos o borrar registros contables.
- **Detalle de Transacciones (El Flujo de la Orden y Mesas):**
  - **Transacción POS y Asignación:** Reemplazando por completo la libreta de papel, el nuevo flujo demanda que el cajero construya la orden mediante un panel interactivo con categorías. El modelo de datos requerirá que, antes de proceder al cobro, la colección del carrito de compras (compuesta por los ítems seleccionados y sus cantidades) sea vinculada obligatoriamente al nombre del cliente y al **número de mesa**.
  - **Facturación y Cierre:** Una vez que se confirma el método de pago, el *back-end* debe procesar matemáticamente el subtotal, aplicar los impuestos correspondientes y sellar la transacción con la fecha exacta. El registro resultante en la base de datos se vuelve inmutable y dispara en el *frontend* la opción de generar e imprimir la factura o ticket (*Bill/Receipt*), dando por finalizado el servicio para esa mesa.

1.11 Propuesta de solución

- **Sistema:** Sistema de Punto de Venta (POS) Web enfocado en TPS.
- **Arquitectura:** Patrón Cliente — Servidor con separación de responsabilidades API REST y arquitectura de tres capas (presentación, lógica de negocio y datos) siguiendo el patrón MVC (Modelo-Vista-Controlador).

Tecnologías seleccionadas (Pila MERN):

Capa	Tecnología	Justificación
Frontend	React.js 18	Biblioteca declarativa con re-renderizado eficiente (Virtual DOM), ideal para interfaces POS reactivas en tiempo real.
Estilos	Tailwind CSS	Framework *utility-first* que agiliza el diseño de interfaces responsivas y táctiles.
Estado global	Redux Toolkit	Manejo centralizado del estado (carrito de pedidos, sesión de usuario, estado de mesas).
Backend	Node.js + Express.js	Entorno no bloqueante basado en eventos, eficiente para múltiples peticiones concurrentes.
Base de datos	MongoDB + Mongoose	Base de datos NoSQL documental, flexible para catálogos dinámicos y estructuras de órdenes variables.
Autenticación	JWT	Autenticación *stateless* y segura, compatible con arquitecturas RESTful y control de roles por *middleware*.
Facturación	PDFKit / jsPDF	Generación programática de recibos y facturas en PDF desde el servidor Node.js.

Capa	Tecnología	Justificación
Contenedores	Docker + Docker Compose	Estandarización del entorno de ejecución entre máquinas del equipo.
Despliegue	AWS EC2 / DigitalOcean	Infraestructura *cloud* confiable con alta disponibilidad y escalado bajo demanda.
Control de versiones	Git + GitHub	Gestión del código fuente con ramas por *feature*, facilitando el trabajo paralelo del equipo.

Tabla 1: Tecnologías seleccionadas (Pila MERN)

A continuación se presenta el diagrama de la arquitectura propuesta:

Diagrama 1: Diagrama de Arquitectura MERN de tres capas del Sistema POS

1.12 Cronograma

El proyecto tiene una duración total de **4 meses (16 semanas)**, organizado en 8 *Sprints* de 2 semanas cada uno bajo el marco Scrum.

Sprint	Semanas	Fase	Actividades principales	Entregable
0	1–2	Inicio y Diseño	Levantamiento de requerimientos, diseño de *wireframes* UI/UX, modelado de base de datos, configuración del repositorio GitHub y entorno Docker.	*Product Backlog*, diseño de BD, *wireframes* aprobados.
1	3–4	*Backend* – Fundamentos	Configuración de Express.js, conexión MongoDB con Mongoose, modelos de datos (User, Product, Table, Order), sistema de autenticación JWT y *middleware* de roles.	API de autenticación funcional (registro, *login*, roles).
2	5–6	*Backend* – Módulos Core	APIs RESTful para gestión de productos del menú (CRUD), gestión de mesas (CRUD + estados) y gestión de órdenes (crear, actualizar estado).	*Endpoints* de Products, Tables y Orders documentados.
3	7–8	*Frontend* – Base y Auth	Configuración de React.js + Redux Toolkit + React Router, pantallas de *Login*, *layout* principal y conexión con el API de autenticación.	*Frontend* base funcional con *login* y roles.
4	9–10	*Frontend* – POS	Módulo POS táctil: selección de categorías, productos, cantidad y adición al carrito de órdenes; envío de pedidos al *backend*.	Interfaz POS funcional conectada al *backend*.

Sprint	Semanas	Fase	Actividades principales	Entregable
5	11–12	Mesas y Dashboard	Panel visual de estados de mesas, módulo de administración de menú (alta/baja de productos), vista de órdenes activas para cocina.	Gestión de mesas e interfaz de cocina operativa.
6	13–14	Facturación y Pagos	Módulo de generación de facturas en PDF, cálculo automático de totales, integración de métodos de pago simulados, historial de ventas.	Facturación y cierre de órdenes completo.
7	15–16	Cierre: QA y Despliegue	Pruebas funcionales e integración (QA), corrección de *bugs*, despliegue en AWS/DigitalOcean con Docker, documentación técnica final y capacitación al usuario.	Sistema desplegado en producción y manual de usuario.

Tabla 2: Cronograma de Sprints

Capítulo 2. MARCO TEÓRICO DEL SISTEMA TPS

2.1 Sistemas de Información Organizacional

2.1.1 Definición

Un Sistema de Información Organizacional (SIO) es un conjunto integrado de componentes — personas, procesos, datos, *hardware* y *software* — diseñado para recolectar, almacenar, procesar y distribuir información que apoye la coordinación, el control, el análisis y la toma de decisiones dentro de una organización (Laudon & Laudon, 2020). A diferencia de un simple programa informático, un SIO está profundamente imbricado con los procesos de negocio de la organización: define cómo fluye la información entre los actores, cuándo se captura, cómo se transforma y quién tiene acceso a ella.

En términos más precisos, un SIO transforma datos crudos (ej. el registro de una venta) en información significativa y estructurada (ej. un reporte de ingresos diarios), que a su vez se convierte en conocimiento útil para la gestión (ej. la identificación del turno de mayor rentabilidad). Este proceso de transformación es el núcleo del valor que aportan los SIO a las organizaciones modernas.

2.1.2 Componentes y Tipos

Todo Sistema de Información Organizacional se articula en torno a seis componentes fundamentales que trabajan de forma interdependiente:

- **Hardware:** La infraestructura física que sustenta el sistema: servidores, terminales de trabajo, dispositivos de red y, en el contexto del presente proyecto, las estaciones de trabajo desde las que el personal operará la interfaz POS.
- **Software:** Los programas y aplicaciones que procesan los datos. Incluye tanto el *software* de sistema (sistema operativo, entorno de ejecución Node.js) como el *software* de aplicación desarrollado a medida (la plataforma POS web).
- **Datos:** La materia prima del sistema. En el contexto de la cafetería, los datos

son las órdenes, los productos, los usuarios, las mesas y las transacciones que el sistema captura y persiste en la base de datos MongoDB.

- **Redes y telecomunicaciones:** La infraestructura de conectividad que permite el acceso concurrente al sistema desde múltiples dispositivos, habilitado por la arquitectura cliente-servidor del proyecto.
- **Procedimientos:** Los protocolos y flujos de trabajo que definen cómo deben interactuar los usuarios con el sistema (ej. el proceso de apertura de turno, la toma de una orden, el cierre de caja).
- **Recursos humanos:** Los actores que operan el sistema. En el proyecto, esto comprende al Administrador y al Cajero, cada uno con roles y permisos claramente delimitados.

Desde una perspectiva funcional, los SIO se clasifican en distintos tipos según el nivel organizacional al que sirven. Los **Sistemas de Procesamiento de Transacciones (TPS)** operan en el nivel operativo, capturando y procesando las transacciones cotidianas del negocio. Los **Sistemas de Información Gerencial (MIS)** consolidan la información del TPS para generar reportes estructurados destinados a la gerencia media. Los **Sistemas de Soporte a Decisiones (DSS)** asisten en la toma de decisiones complejas mediante análisis de datos y modelos. Los **Sistemas de Información Ejecutiva (EIS)** proveen información estratégica de alto nivel a los directivos. El presente proyecto se enfoca en la implementación de un TPS, que actúa como la base de toda esta pirámide informacional.

2.1.3 Importancia en organizaciones del sector gastronómico

En establecimientos de servicio de alimentos y bebidas, los SIO basados en TPS son particularmente críticos debido al alto volumen de transacciones de bajo valor unitario que se procesan en intervalos de tiempo muy cortos. La velocidad, exactitud y trazabilidad de cada transacción impactan directamente en la experiencia del cliente, en el control de ingresos y en la capacidad de la gerencia para gestionar el negocio de manera sostenible. La implementación de un TPS en este contexto no es un lujo tec-

nológico, sino una condición de competitividad operativa.

2.2 Sistema de Procesamiento de Transacciones (TPS)

Un Sistema de Procesamiento de Transacciones es un tipo especializado de SIO diseñado para capturar, procesar, validar y almacenar las transacciones operativas de una organización de forma inmediata, confiable y a gran escala (O'Brien & Marakas, 2011). En el contexto del negocio, una **transacción** se define como cualquier evento discreto que modifica el estado de los datos del sistema y que debe quedar registrado de forma permanente e inalterable: el registro de una venta, la creación de una orden, el cobro de una cuenta o la modificación del catálogo de productos.

2.2.1 Características principales

Los TPS se distinguen de otros tipos de sistemas de información por un conjunto de atributos técnicos y funcionales que los hacen aptos para el procesamiento operativo de alto volumen:

- **Procesamiento en tiempo real (*OLTP*):** A diferencia del procesamiento por lotes (*batch*), los TPS modernos procesan cada transacción en el instante en que se produce, actualizando la base de datos de forma inmediata y reflejando el estado actual del negocio en todo momento.
- **Alta confiabilidad y disponibilidad:** Un TPS para un punto de venta debe estar disponible durante todo el horario operativo del negocio. La indisponibilidad del sistema implica la parálisis del servicio al cliente.
- **Integridad transaccional (propiedades ACID):** Toda transacción en un TPS debe cumplir las propiedades de Atomicidad (la transacción se ejecuta completa o no se ejecuta), Consistencia (el sistema pasa de un estado válido a otro estado válido), Aislamiento (las transacciones concurrentes no interfieren entre sí) y Durabilidad (una transacción confirmada persiste incluso ante fallos del sistema).
- **Manejo de alto volumen de datos estandarizados:** Los TPS están optimizados para procesar grandes cantidades de transacciones simples y repetitivas de

forma eficiente, a diferencia de los sistemas analíticos que trabajan con consultas complejas sobre datos históricos.

- **Generación de documentos y comprobantes:** Una función esencial del TPS es la emisión automática de comprobantes (tickets, facturas, recibos) que documentan cada transacción para el cliente y para los registros contables del negocio.

2.2.2 Funciones del TPS en el sistema POS de la cafetería

En el contexto específico del presente proyecto, el TPS ejecuta el siguiente ciclo funcional para cada transacción de venta:

1. **Captura de datos de origen:** El cajero construye la orden seleccionando productos del catálogo digital y asignándola a una mesa, introduciendo los datos de la transacción en el sistema mediante la interfaz POS de React.js.
2. **Validación y verificación:** El *backend* (Node.js/Express.js) verifica que el usuario tenga los permisos necesarios (validación JWT), que los productos existan en el catálogo activo y que la mesa esté disponible.
3. **Procesamiento matemático:** El motor transaccional calcula automáticamente los subtotales por ítem, aplica los impuestos correspondientes y determina el total a cobrar, eliminando el margen de error del cálculo manual.
4. **Actualización de la base de datos:** La transacción se escribe de forma atómica en MongoDB, vinculando el documento de la orden con el cajero responsable, la mesa asignada y los ítems del carrito con sus precios exactos en ese instante.
5. **Emisión del comprobante:** El sistema genera el ticket o factura en formato PDF, disponible para impresión inmediata, y actualiza el estado de la mesa a “disponible”.

2.2.3 Evolución hacia sistemas web

Los TPS han recorrido un largo camino desde las terminales monolíticas de los años setenta. La adopción de arquitecturas web modernas —como la empleada en este proyecto— representa la fase más reciente de esta evolución, caracterizada por tres

ventajas fundamentales: **ubicuidad** (el sistema es accesible desde cualquier dispositivo con navegador web en la red local del negocio), **centralización** (todos los datos residen en un único repositorio en la nube, eliminando la dispersión de información), y **escalabilidad** (la arquitectura basada en microservicios y contenedores Docker permite escalar el sistema horizontalmente para absorber incrementos en la carga de trabajo sin rediseñar la arquitectura base).

2.3 Arquitectura de sistemas web

- **Cliente (*Frontend*):** La interfaz que interactúa con el usuario final a través de un navegador web, responsable de la presentación de la información (Ej. Pantalla POS de React).
- **Servidor (*Backend*):** La computadora central o contenedor de la nube que procesa la lógica de negocios, gestionando la concurrencia y los cálculos matemáticos.
- **API (*Application Programming Interface*):** El puente de comunicación documentado entre el Cliente y el Servidor (vía JSON).
- **Base de datos:** Repositorio central donde reposa la persistencia de las entidades.

2.4 Seguridad en sistemas de información

Como modelador y encargado de la seguridad arquitectónica, se establece que un sistema de ventas (POS) debe proteger de forma absoluta sus *endpoints* (rutas de API) y la persistencia de datos.

- **Autenticación mediante JWT:** Se descartan las sesiones tradicionales. El sistema implementa JSON Web Tokens (JWT). Tras validar credenciales (contraseñas previamente procesadas con funciones criptográficas unidireccionales de *hash*, como *bcrypt*), el *backend* emite un token firmado. Este token viaja en las cabeceras HTTP de cada petición del cliente, garantizando que el usuario es quien dice ser sin consultar la base de datos reiteradamente.

- **Roles y Autorización:** El modelo de datos incluye una propiedad rígida de “Rol” (ej. Administrador, Cajero). La autorización se ejecuta mediante *Middlewares* (bloques de código intermedios en el *backend*) que desencriptan el *payload* del token y rechazan con un error 403 (Prohibido) cualquier intento de un Cajero de acceder a las rutas de eliminación de usuarios o reportes gerenciales.
- **Control de acceso:** Tanto a nivel de la interfaz (ocultando botones de configuración a cajeros) como a nivel de capa de datos, se aplican políticas estrictas para evitar inyecciones maliciosas o robo de sesiones, blindando el flujo desde que se presiona “Cobrar” hasta que la información reposa en el disco.

2.5 Base de datos

El diseño de la persistencia de datos constituye el corazón del sistema, siendo responsabilidad directa de la ingeniería de datos modelar la información de la cafetería para que sea escalable, rápida y matemáticamente exacta. Basado en la arquitectura MERN (MongoDB, Express, React, Node.js), se adopta un enfoque orientado a documentos que optimiza la velocidad transaccional en el Punto de Venta:

- **Modelo Documental (NoSQL) y Referencias Lógicas:** Al utilizar MongoDB, la base de datos abandona las tablas rígidas en favor de colecciones de documentos JSON (BSON). Sin embargo, los principios lógicos de integridad se mantienen ineludibles mediante el uso de esquemas de validación estrictos (como *Mongoose*). Las colecciones maestras se identifican y separan claramente: `Usuarios` (Personal operativo), `Categorías` (Clasificación del menú), `Productos` (Ítems de venta), `Mesas` (Gestión del espacio físico) y `Facturas/Órdenes` (*Bills* - Registro de la transacción). Se definen referencias explícitas (a través de *ObjectIds*) entre ellas para establecer la cardinalidad (ej. una Orden está vinculada unívocamente a un Cajero y asignada a una Mesa específica).
- **Desnormalización Controlada y Estructura del Carrito:** A diferencia de un modelo relacional tradicional, MongoDB aprovecha la desnormalización para mejorar el rendimiento de lectura. Por requerimientos de diseño de un POS, se

realiza una desnormalización controlada en las `Órdenes`: al registrar una venta, no solo se guarda la referencia del producto, sino que se incrusta el arreglo completo del carrito (`cartItems`). El nombre de la bebida, sus modificadores (ej. tamaño, adiciones) y su **precio exacto en ese instante** se copian dentro del documento de la orden. Esto garantiza que la información histórica sea inmutable frente a futuros cambios de costos en el catálogo.

- **Transacciones Multi-documento y Aislamiento (ACID):** En el entorno TPS, una transacción es indivisible. Registrar una venta en la cafetería implica una cadena de eventos: calcular totales, insertar el documento de la factura, asociar el método de pago y actualizar el estado de disponibilidad de la `Mesa`. El motor de MongoDB se configura utilizando *Sesiones de Transacción* para garantizar las propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad) a través de múltiples colecciones. Esto asegura que si ocurre un fallo de red o desconexión a la mitad del proceso de cobro, la base de datos ejecute un *Rollback* (reversión completa), previniendo que existan "ventas a medias" o corrupciones en los arqueos de caja.

2.6 Metodología de desarrollo

Scrum Es un marco de trabajo ágil para el desarrollo, entrega y mantenimiento de productos complejos, definido en la ***Scrum Guide*** (Schwaber & Sutherland). Se fundamenta en tres pilares empíricos: **transparencia** (todos los aspectos del proceso son visibles para los responsables), **inspección** (los artefactos y el progreso son revisados frecuentemente) y **adaptación** (el proceso se ajusta cuando se detectan desviaciones). Para el Sistema POS de Cafetería, Scrum es la elección metodológica óptima porque el cliente puede proporcionar retroalimentación rápida sobre cada incremento funcional, y los requerimientos pueden evolucionar durante el desarrollo.

2.6.1 Roles

Product Owner — Torrez Calle Álvaro Ariel

El *Product Owner* es el responsable de maximizar el valor del producto. Sus funciones

concretas en este proyecto son: definir, ordenar y mantener el *Product Backlog* (lista priorizada de funcionalidades); ser el punto de contacto único con el cliente (la cafetería); aceptar o rechazar los incrementos al final de cada *Sprint*; y garantizar que el equipo de desarrollo comprenda los ítems del *backlog* al nivel necesario.

Scrum Master — Maldonado Carvajal Alan Ariel

El *Scrum Master* es el responsable de que el equipo comprenda y aplique correctamente Scrum. Sus funciones son: facilitar las ceremonias (*Planning*, *Daily*, *Review*, *Retrospective*); eliminar impedimentos que bloqueen al equipo de desarrollo; proteger al equipo de interrupciones externas durante el *Sprint*; y ayudar al *Product Owner* a mantener un *backlog* ordenado y comprensible.

Equipo de Desarrollo — Claros Suntura Juan José y Lecoña Condori Elías Milán

El Equipo de Desarrollo es autoorganizado y multifuncional. Es responsable de convertir los ítems del *Product Backlog* seleccionados en un incremento potencialmente entregable al final de cada *Sprint*. Sus funciones incluyen estimar el esfuerzo de las historias de usuario, codificar el *frontend* y *backend*, y ejecutar pruebas de integración.

2.6.2 Artefactos

Product Backlog (Pila de producto): Lista única, ordenada y emergente de todo lo que se necesita en el producto, gestionada exclusivamente por el *Product Owner*. A continuación se presentan los ítems iniciales:

ID	Historia de Usuario	Prioridad	Puntos
US-01	*Como* mesero, *quiero* iniciar sesión con mis credenciales *para* acceder solo a las funciones de mi rol.	Alta	3

ID	Historia de Usuario	Prioridad	Puntos
US-02	*Como* cajero, *quiero* ver el panel de mesas y su estado actual *para* saber cuáles están ocupadas.	Alta	5
US-03	*Como* mesero, *quiero* agregar productos al pedido de una mesa desde la interfaz POS táctil.	Alta	8
US-04	*Como* administrador, *quiero* registrar y editar los productos del menú con sus precios.	Alta	5
US-05	*Como* cajero, *quiero* generar e imprimir la factura de una orden cerrada en PDF.	Media	8
US-06	*Como* administrador, *quiero* ver el historial de ventas del día con totales por método de pago.	Media	5

Tabla 3: Product Backlog Inicial

Sprint Backlog (Pila del ciclo): Conjunto de ítems del *Product Backlog* seleccionados para el *Sprint* actual, más el plan del equipo para entregar el Incremento y lograr el *Sprint Goal*. Es propiedad del Equipo de Desarrollo y se actualiza diariamente mediante un tablero Kanban con columnas: **Por hacer** → **En progreso** → **En revisión** → **Hecho**.

Incremento del Producto: Suma de todos los ítems completados durante el *Sprint* más el valor de los *Sprints* anteriores. Debe cumplir la **Definición de Hecho (DoD)** acordada por el equipo, la cual establece que un ítem está "hecho" cuando el código está *commiteado* en la rama correspondiente de GitHub, las pruebas funcionales han pasado, el código ha sido revisado por al menos otro miembro del equipo (*code review*) y la funcionalidad ha sido demostrada al *Product Owner*.

2.6.3 Eventos

Sprint Planning* (Planificación del *Sprint*):** Se realiza al inicio de cada *Sprint* (duración máxima: 4 horas). El equipo completo responde dos preguntas: ¿qué se puede entregar en este *Sprint*? (selección del *Sprint Backlog*) y ¿cómo se logrará ese trabajo? (descomposición en tareas técnicas). Al final se define el ***Sprint Goal: el objetivo en una frase que da coherencia a los ítems seleccionados.

***Daily Scrum* (Scrum Diario):** Reunión de 15 minutos, todos los días, para el Equipo de Desarrollo. Cada integrante responde: ¿qué hice ayer que ayudó al *Sprint Goal*? ¿Qué haré hoy? ¿Hay algún impedimento? El *Scrum Master* facilita y anota los impedimentos para resolverlos de inmediato.

***Sprint Review* (Revisión del *Sprint*):** Se realiza al final del *Sprint* (duración máxima: 2 horas). El Equipo de Desarrollo demuestra el Incremento funcionando al *Product Owner* y *stakeholders*. Se actualiza el *Product Backlog* en función de la retroalimentación recibida.

***Sprint Retrospective* (Retrospectiva del *Sprint*):** Se realiza después de la *Sprint Review* (duración máxima: 1.5 horas). El equipo reflexiona sobre el proceso: ¿qué salió bien? ¿qué se puede mejorar? ¿qué acciones concretas tomamos en el próximo *Sprint*? El resultado es un plan de mejora que se incorpora al siguiente *Sprint Backlog*.

Capítulo 3. MARCO PRÁCTICO — DESARROLLO DEL SISTEMA TPS

3.1 Análisis del sistema

En base a la recopilación de datos, la estructura organizacional del sistema de la cafetería identifica actores con responsabilidades asimétricas: Administradores, enfocados en la gobernanza del catálogo y métricas; y Cajeros, cuyo objetivo es el alto rendimiento en el registro y cobro de pedidos. El flujo exige digitalizar cada interacción, desde la selección del producto táctil hasta su asignación a la mesa y el cobro final.

Figura 1: Ejemplo de prueba en el análisis del sistema

3.2 Determinación de requerimientos

3.2.1 Requerimientos funcionales

Los requerimientos funcionales expresan lo que el sistema **debe hacer** operativamente.

- **RF01:** El sistema permitirá autenticar de forma segura a cajeros y administradores.
- **RF02:** El sistema proporcionará una interfaz de POS para la selección ágil de productos del menú por categoría.
- **RF03:** El sistema permitirá asignar obligatoriamente un número de “Mesa” a cada carrito de compra.
- **RF04:** El sistema calculará automáticamente subtotales, impuestos y totales conforme se añaden o restan productos.
- **RF05:** El sistema permitirá a los administradores gestionar colecciones maestras (Productos, Categorías, Usuarios).
- **RF06:** El sistema generará e imprimirá comprobantes (tickets) al concretar el cobro.

3.2.2 Requerimientos no funcionales

Establecen las restricciones y la forma en cómo debe operar y comportarse estructuralmente la aplicación.

- **Seguridad:** Encriptación de contraseñas usando algoritmos seguros (*bcrypt*) y protección de rutas con JWT.
- **Rendimiento:** Tiempos de procesamiento del carrito y confirmación de pago menores a 1 segundo para no detener la fila.
- **Usabilidad:** Interfaz basada en principios POS: botones amplios e interacciones con mínimos clics.
- **Escalabilidad:** Separación modular estricta entre *Frontend* (React) y *Backend* (Node.js).

3.3 Modelado del sistema

3.3.1 Historias de Usuario

Se definen detallando la necesidad y la regla de aceptación comercial:

- *Como* administrador de la cafetería, *quiero* gestionar los productos y sus precios *para* mantener el menú actualizado.
- *Como* cajero en turno, *quiero* agregar bebidas al ticket y asignarlo a una mesa mediante la pantalla *para* atender ágilmente a los clientes y evitar confusiones en sala.

3.3.2 Diagramas UML

A continuación se muestra un ejemplo genérico de la estructura de un diagrama, en este caso, se deben incrustar aquí los diagramas correspondientes: Diagramas de Casos de Uso, Clases/Esquemas NoSQL, y Actividades.

Diagrama 2: Ejemplo general de Diagrama Estructural UML

3.4 Diseño del sistema

3.4.1 Arquitectura del sistema

Modelo: Arquitectura Web Cliente-Servidor (Pila MERN) El sistema se dividirá lógicamente en una aplicación cliente *Single Page Application* (React) interactuando asíncronamente con un servidor *backend* (Node.js/Express) que expondrá puntos de enlace (*endpoints*) REST y conectará con el motor de base de datos.

3.5 Diseño de la Base de Datos

Se diseña bajo el paradigma documental (NoSQL) garantizando el aislamiento financiero. Las restricciones en el esquema aseguran que una “Orden” sea indivisible e incorpore el carrito completo para inmutabilidad histórica.

Campo (Documento JSON)	Tipo	Validaciones / Vínculos	Descripción
_id	ObjectId	Único (PK lógica)	Identificador nativo de MongoDB para la factura.
usuario_id	ObjectId	Ref: ’Usuario’	ID del cajero responsable de cobrar la orden.
mesa_asignada	String/Int	Requerido	Mesa física vinculada al pedido.
cartItems	Array	Contiene Objetos	Arreglo desnormalizado con los productos, cantidades y precios exactos en el momento de venta.
total_pagado	Number	Mínimo 0	Monto económico final calculado y liquidado.

Tabla 4: Diccionario para colección de Órdenes (Base de Datos)

3.6 IMPLEMENTACIÓN DE LOS MÓDULOS DEL SISTEMA

3.6.1 Módulo de Catálogo, Menú y Mesas

Módulo fundamental para configurar el entorno de la cafetería.

- **Gestión de colecciones:** Interfaces CRUD para ingresar categorías, ítems del menú con sus precios, y la estructura de las mesas del local.
- **Estados de visualización:** Activación y desactivación de productos agotados sin borrarlos de la base de datos.

3.6.2 Módulo de Usuarios y Roles

La barrera de seguridad criptográfica del sistema TPS.

- **Filtro Criptográfico:** Validaciones de contraseña cruzadas con *bcrypt* en el inicio de sesión.
- **Control de acceso (RBAC):** El *frontend* renderizará la “Terminal POS” para los cajeros y bloqueará las pestañas de “Administración”, regla que es respaldada por los *middlewares* de validación de *tokens* JWT en el servidor.

3.6.3 Módulo de Transacciones (El Punto de Venta)

Núcleo central del objeto TPS, diseñado para alta velocidad operativa.

- **Interfaz de Venta:** Pantalla con cuadrícula de productos a la izquierda y el carrito reactivo a la derecha.
- **Motor Matemático:** Sumatorias dinámicas (Redux) inyectables a la API para certificar la venta atómica.
- **Cobro y Cierre:** Ventana modal que captura el método de pago, asigna la mesa, y consolida irrevocablemente el documento JSON en MongoDB, disparando la generación del ticket.

3.6.4 Módulo de Reportes

Módulo analítico que destila la información transaccional operativa del local.

- Extracción de ingresos consolidados mediante consultas de agregación (*Aggregations*) en MongoDB.
- Filtros por usuario y fechas para realizar el cierre y arqueo de caja exacto al final del turno.

3.7 Capa Backend Funcional

El *backend* es el cerebro transaccional aislado de la interfaz gráfica, diseñado bajo principios REST:

- **Controladores (*Controllers*):** Capturan las peticiones HTTP (req, res).
- **Modelos/Esquemas (*Mongoose Models*):** Representación estricta de la estructura de los documentos en la base de datos (Ej. `BillSchema`, `ItemSchema`).
- **Capa de Seguridad (*Middlewares*):** Componentes (como `verifyToken`) que inspeccionan las cabeceras HTTP antes de conceder cualquier inserción a la base de datos.
- **Lógica Transaccional:** Uso de sesiones para asegurar que el guardado de la factura y la limpieza de la mesa se ejecuten como una transacción ACID única.

3.8 Validación y pruebas del sistema

El sistema asegura la calidad del producto final a través de distintas evaluaciones:

- **Pruebas de Estrés Unitarias:** Validar que el servidor sume grandes carritos de compra sin errores de formato decimal.
- **Pruebas funcionales de reversión:** Simular el error de la red justo en el momento de procesar un pago para verificar que la transacción se deshaga (*Rollback*) correctamente.

- **Pruebas de aceptación:** Pruebas finales en la caja registradora física para certificar la fluidez del *software*.

3.9 Desarrollo del prototipo funcional

A lo largo de los *sprints* se generan prototipos incrementales exponiendo sus interfaces interactivas reflejando el caso de éxito: desde el *login* del cajero, armado de la orden, selección de mesa, hasta el cobro. *(Incluir evidencias y capturas de pantalla reales aquí)*.

3.10 Documentación de Ingeniería Completa

Se acompañan los anexos técnicos:

- **Documentación funcional:** Relevamiento documentado de requerimientos e historias de usuario de la cafetería.
- **Documentación técnica:** Modelo arquitectónico, diseño de esquemas BSON (Colecciones) y documentación de *endpoints* (API REST).
- **Documentación del sistema:** Manual de usuario, variables de entorno y directrices de despliegue.

Capítulo 4. CONCLUSIONES Y RECOMENDACIONES

4.1 Conclusiones

- Tras el proceso de desarrollo, se implementó satisfactoriamente el Sistema de Punto de Venta (POS) Web basado en el enfoque TPS para la cafetería, abordando con éxito el reemplazo de comandas manuales por un entorno digital unificado.
- La adopción de la pila MERN (MongoDB, Express, React, Node.js) y el diseño de persistencia orientada a documentos demostraron ser ideales para optimizar la velocidad de escritura del carrito de compras y garantizar la inmutabilidad de los reportes históricos.
- La seguridad del sistema fue robustecida a través del esquema de autenticación JWT y encriptación de credenciales, consolidando un TPS que erradica los descuadres de caja y facilita la gestión de mesas y auditorías administrativas.

4.2 Recomendaciones

- **Mejoras futuras y Escalabilidad:** Es recomendable planificar la ampliación del módulo transaccional para descontar automáticamente ingredientes de un inventario maestro (ej. gramos de café, litros de leche) cada vez que se cobre un producto.
- **Seguridad y Mantenimiento:** Monitorear activamente los registros del servidor, rotar periódicamente la clave secreta de firmado de los *tokens* JWT y actualizar las dependencias de Node.js para prevenir vulnerabilidades.
- **Migración a la Nube:** Aprovechando la naturaleza NoSQL de MongoDB, se recomienda a futuro desplegar la base de datos en clústeres administrados (como MongoDB Atlas) para asegurar redundancia, *backups* automáticos y la posibilidad de conectar múltiples sucursales de la cafetería.